

Diffusion Model

2026 年 9 月 23 日

1 引言

在很长一段时间里，我对 AI 生图的印象还停留在“风格化渲染”上：给模型一张照片，它返回一张动漫风格的图。直到 GPT 2.0 系列模型出现，这个印象才被彻底打破——它生成的 90 年代中国家庭照片逼真得令人惊讶，构图、光线乃至老照片特有的质感都恰到好处（图 1）。



图 1: AI 生图示例

于是我开始认真了解图像生成这个方向，很快发现一个名字反复出现：Diffusion Model。今天图像生成领域的主流工作几乎都建立在它之上；而在机器人学习领域，也有一批颇有影响力的工作（例如 Diffusion Policy）以它为骨架。市面上有很多关于扩散模型的博客与视频，但是他们大多都有些复杂亦或是不够直观，于是我就想尝试写一篇关于扩散模型的博客，以便日后在自己忘记相关知识时回头看能再次捡起。

本文主要梳理了扩散模型的基本思想，并尝试完整推导 DDPM 的训练目标与采样过程，侧重于直观理解，省略了部分纯计算性的细节，并在其中附上了笔者自己的理解，如有错误，还请指正；整体框架参考了李宏毅《机器学习》2023 春季课程中扩散模型的相关章节。

2 基本概念

先从一个形象的例子开始。和所有机器学习方法一样，扩散模型方法无法输出它从未见过的样本：训练时需要一批真实图像，最终生成的也只是无限逼近这批样本。比如用一批猫的照片训练，模型就生成不出狗，而只能生成猫，或者猫的组合——这里说的组合是指：比如用白猫与黑猫的照片训练后，模型可以生成一只头像白色、身子纯黑的猫，如图 2 所示。当然现在网上已经有很多这样的组合，比如将某些明星 P 图在饮料上亦或是将某些动漫卡通图像从它原本的形态生成一张青蛙形态的照片。



白猫

黑猫

合成猫

图 2: 训练与生成

扩散模型可以拆成方向相反的两条链：前向过程往真实图片里不断加入噪声，直到它接近标准高斯噪声；反向过程反过来走，从随机噪声出发一步步去除之前加入的噪声，最终恢复出图片。

两条链的流程分别如图 3 与图 4 所示。加噪链上，每一步都在当前照片上叠加一次高斯噪声（箭头上方的噪声图块就是这一步加入的 ϵ ），加噪 n 次之后照片变成一片纯噪声 x_T ；去噪链则把同一组照片倒过来走，每一步都由网络预测当前混入的噪声、再减掉一部分，去噪 n 次之后恢复出干净的 x_0 。

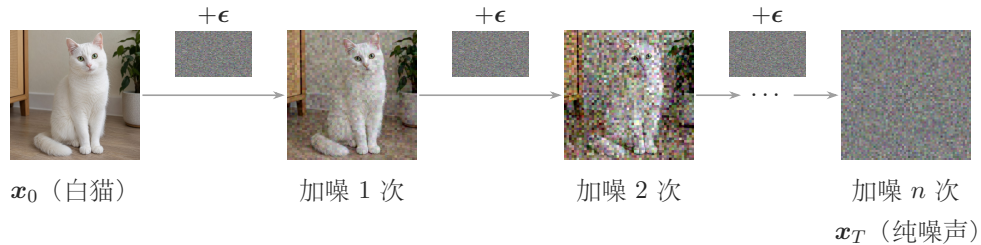


图 3: 前向加噪流程

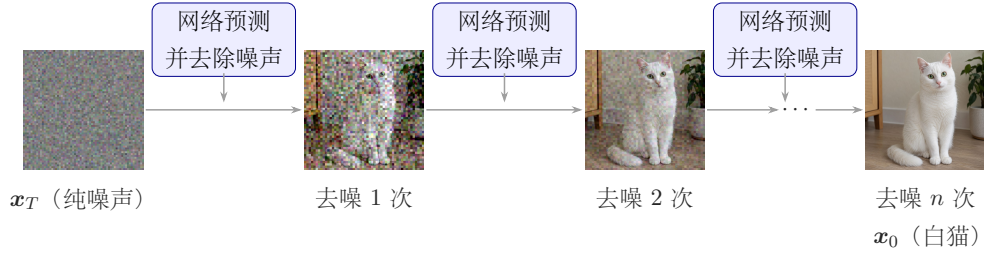


图 4: 反向去噪流程

这个过程很像还原一个魔方：前向过程是人为地把一个六面同色的标准魔方一步步打乱，直到六个面全都乱掉；反向过程则是沿相反的轨迹把它拧回去，直到恢复成标准魔方。当然，现实中的魔方有现成公式可循，以及现实生活中我们的标准状态并非是六面同色的状态，比如现在有很多博主尝试用多个魔法去画心形亦或是其他图形。但这里关心的不是公式，而是让模型记住我们想要的“标准状态”是什么，再把打乱后的魔方交还给它去复原回去。

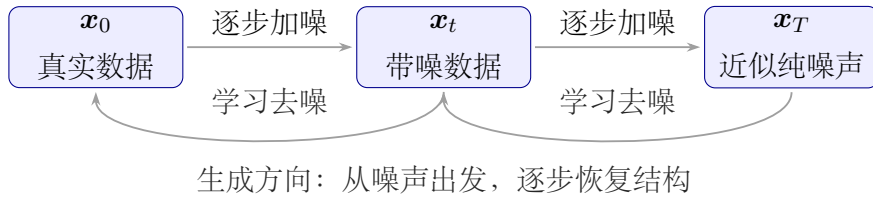


图 5: 前向与反向

3 前向加噪

3.1 单步加噪

设有一张干净、清晰的图片作为初始样本，记为 $\mathbf{x}_0$ 。前向过程规定了一条固定的马尔可夫链：每一步只在当前图片上叠一点点高斯噪声，加多少由系数 $\beta_t \in (0, 1)$ 决定。 $\mathbf{x}_t$ 、 $\mathbf{x}_{t-1}$ 与这一步加入的噪声 ϵ_t 之间的关系式就是

$$\mathbf{x}_t = \sqrt{1 - \beta_t} \mathbf{x}_{t-1} + \sqrt{\beta_t} \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \quad (3.1)$$

换句话说，在给定 $\mathbf{x}_{t-1}$ 时， $\mathbf{x}_t$ 服从一个均值为 $\sqrt{1-\beta_t}\mathbf{x}_{t-1}$ 、方差为 $\beta_t\mathbf{I}$ 的高斯分布：

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}\left(\sqrt{1-\beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I}\right), \quad t = 1, 2, \dots, T. \quad (3.2)$$

其中 β_t 是第 t 步的噪声强度，它随着 t 的增大而增大：也就是说，随着噪声不断加入，图像里噪声的占比越来越大、原来真实图像的占比越来越小，直到最终的 $\mathbf{x}_T$ 中几乎全部都是噪声。这两个系数的取法可以严格说清楚，出发点只有一条：让加噪之后样本的边缘方差保持不变。为此需要一条关于高斯的基本结论：若 $X \sim \mathcal{N}(\mu_1, \sigma_1^2)$ 与 $Y \sim \mathcal{N}(\mu_2, \sigma_2^2)$ 相互独立，则它们的加权和仍是高斯，且

$$aX + bY \sim \mathcal{N}(a\mu_1 + b\mu_2, a^2\sigma_1^2 + b^2\sigma_2^2). \quad (3.3)$$

从定义可以直接验证：期望按线性叠加，是 $a\mu_1 + b\mu_2$ ；方差为 $a^2\sigma_1^2 + b^2\sigma_2^2 + 2ab\text{Cov}(X, Y)$ ，独立性使协方差项恰好为零（从密度的角度看，这等价于两个高斯密度做卷积，而卷积结果仍是高斯密度）。

把式 (3.3) 用在式 (3.1) 上（取 $a = \sqrt{1-\beta_t}$ 、 $b = \sqrt{\beta_t}$ ，两个噪声项均值为零、方差为 1），得到 $\mathbf{x}_t$ 的边缘方差：

$$\text{Var}(\mathbf{x}_t) = (1 - \beta_t)\text{Var}(\mathbf{x}_{t-1}) + \beta_t. \quad (3.4)$$

这是一个把方差往 1 收拢的不动点迭代：若 $\text{Var}(\mathbf{x}_{t-1}) = 1$ ，则 $\text{Var}(\mathbf{x}_t) = 1$ ；方差偏大就被压小，偏小就被抬大。因此只要初始数据做过标准化（实际训练正是如此），整条链上每一步的边缘方差都严格等于 1，加噪再多也不会尺度爆炸或塌缩。顺带这也解释了根号从哪里来：方差里的贡献是系数的平方，要让旧内容的方差贡献恰好乘 $(1 - \beta_t)$ 、新噪声恰好贡献 β_t ，两个系数就必须开根号；若均值系数直接写成 $1 - \beta_t$ ，方差会按 $(1 - \beta_t)^2$ 衰减，几十步后样本尺度就塌掉了。单步的 β_t 通常很小（例如从 10^{-4} 线性增长到 0.02），所以单看一步几乎看不出变化——这正是“扩散”这个名字的由来：像一滴墨在清水里慢慢散开。

3.2 一步到位

实际操作中，并不会真的一步一步加 t 次噪声：既然每一步的关系都已知，就可以把式 (3.1) 逐层代入，直接推出 $\mathbf{x}_t$ 与 $\mathbf{x}_0$ 的关系，让整个前向加噪等效成一步。记

$$\alpha_t = 1 - \beta_t, \quad \bar{\alpha}_t = \prod_{s=1}^t \alpha_s. \quad (3.5)$$

以两步为例，把式 (3.1) 代入自身：

$$\begin{aligned} \mathbf{x}_2 &= \sqrt{\alpha_2} \mathbf{x}_1 + \sqrt{1 - \alpha_2} \boldsymbol{\epsilon}_2 \\ &= \sqrt{\alpha_1 \alpha_2} \mathbf{x}_0 + \underbrace{\sqrt{\alpha_2(1 - \alpha_1)} \boldsymbol{\epsilon}_1 + \sqrt{1 - \alpha_2} \boldsymbol{\epsilon}_2}_{\text{两个独立高斯的加和}}. \end{aligned} \quad (3.6)$$

关键就在最后这一项：它把两次加入的噪声合到了一起。直接套用式 (3.3)——两个噪声相互独立、均值都是零，系数为 $a = \sqrt{\alpha_2(1 - \alpha_1)}$ 与 $b = \sqrt{1 - \alpha_2}$ ，于是加和的均值为零、方差按系数平方累加：

$$\sqrt{\alpha_2(1 - \alpha_1)} \boldsymbol{\epsilon}_1 + \sqrt{1 - \alpha_2} \boldsymbol{\epsilon}_2 \sim \mathcal{N}\left(\mathbf{0}, \underbrace{[\alpha_2(1 - \alpha_1) + (1 - \alpha_2)] \mathbf{I}}_{= 1 - \alpha_1 \alpha_2 = 1 - \bar{\alpha}_2}\right), \quad (3.7)$$

所以这两个噪声可以等效成一个标准形式的高斯，记作 $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 。照此规律归纳到任意 t 步，就得到从 $\mathbf{x}_0$ 直接跳到任意时刻的闭式：

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \quad (3.8)$$

也就是说，前面 t 次分别加入的噪声，合起来等效为一个方差为 $1 - \bar{\alpha}_t$ 的高斯噪声； $\bar{\alpha}_t$ 随 t 单调减小，噪声的等效占比就越来越大，直到 $t = T$ 时 $\mathbf{x}_T$ 近似纯噪声。

关于 β_t 该怎么取：取太小， T 必须很大、采样很慢；取太大，单步破坏的信息过多、反向过程更难学。常见做法是让 β_t 从 10^{-4} 到 0.02 线性增长，或者采用余弦形式的调度。

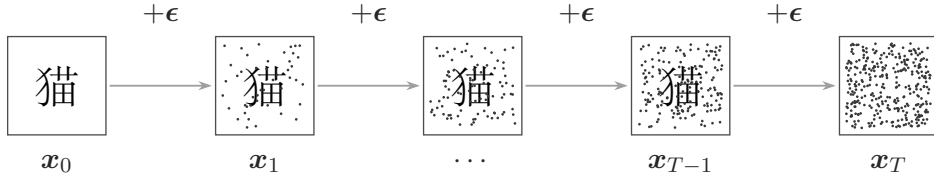


图 6: 前向加噪示意

4 反向去噪

4.1 单步去噪

生成时手里只有 $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ ，希望一步步退回 $\mathbf{x}_0$ 。反向过程同样是一条马尔可夫链，它的每一步是条件分布 $q(\mathbf{x}_{t-1} | \mathbf{x}_t)$ ，直观上回答的问题是：拿到一张加了 t 次噪声的图片，它少加一次噪声时最可能长什么样？

按贝叶斯公式把它展开，

$$q(\mathbf{x}_{t-1} | \mathbf{x}_t) = \frac{q(\mathbf{x}_{t-1}) q(\mathbf{x}_t | \mathbf{x}_{t-1})}{q(\mathbf{x}_t)}. \quad (4.1)$$

分子里的 $q(\mathbf{x}_t | \mathbf{x}_{t-1})$ 就是前向单步，式 (3.2) 已经给出；真正卡住我们的是 $q(\mathbf{x}_{t-1})$ 和分母 $q(\mathbf{x}_t)$ ——它们是数据本身的分布。想把 $q(\mathbf{x}_t)$ 算出来，就要对全体可能的干净图片积分：

$$q(\mathbf{x}_t) = \int q(\mathbf{x}_t | \mathbf{x}_0) q(\mathbf{x}_0) d\mathbf{x}_0, \quad (4.2)$$

而其中的 $q(\mathbf{x}_0)$ 是真实图片的分布——这正是想学、却从来不知道的东西。于是死循环出现了：要写出反向的一步，得先知道整个数据集的分布；可“学会数据集的分布”恰恰是训练扩散模型的目的。

所以结论是： $q(\mathbf{x}_{t-1} | \mathbf{x}_t)$ 并不是不存在，而是没有解析形式，无法像前向过程那样用一个公式直接写出来。它也只能是一个分布而不能是一个函数——同一张 $\mathbf{x}_t$ 本来就可以由许多不同的 $\mathbf{x}_{t-1}$ 加噪得到。既然写不出来，只能让参数为 θ 的神经网络去近似这整个单步反向分布：

$$p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t) = \mathcal{N}(\boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \sigma_t^2 \mathbf{I}). \quad (4.3)$$

但要用好这个网络，得先知道“正确的单步反向分布”长什么样，作为它的学习目标。这就是下一小节要做的事。

4.2 反向分布

直接求解 $q(\mathbf{x}_{t-1} | \mathbf{x}_t)$ 的出路在于：额外附加一个条件 $\mathbf{x}_0$ 。多知道起点之后，贝叶斯公式里出现的每一项都来自前向过程，全部可以写出解析形式。这件事从条件概率的定义出发，一步一步做。

第一步，按条件概率的定义，把左边的后验写成联合分布之比：

$$q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) = \frac{q(\mathbf{x}_{t-1}, \mathbf{x}_t, \mathbf{x}_0)}{q(\mathbf{x}_t, \mathbf{x}_0)}. \quad (4.4)$$

第二步，对分子、分母分别套用乘法公式（链式法则），只是展开的顺序不同：分子从 $\mathbf{x}_0$ 出发沿前向链逐项展开，分母一步到位：

$$q(\mathbf{x}_{t-1}, \mathbf{x}_t, \mathbf{x}_0) = q(\mathbf{x}_0) q(\mathbf{x}_{t-1} | \mathbf{x}_0) q(\mathbf{x}_t | \mathbf{x}_{t-1}, \mathbf{x}_0), \quad (4.5)$$

$$q(\mathbf{x}_t, \mathbf{x}_0) = q(\mathbf{x}_0) q(\mathbf{x}_t | \mathbf{x}_0). \quad (4.6)$$

第三步，把式 (4.5) 与式 (4.6) 代回式 (4.4)：分子分母里的 $q(\mathbf{x}_0)$ 直接约去；剩下的 $q(\mathbf{x}_t | \mathbf{x}_{t-1}, \mathbf{x}_0)$ 由马尔可夫性退化为 $q(\mathbf{x}_t | \mathbf{x}_{t-1})$ ——给定 $\mathbf{x}_{t-1}$ 之后， $\mathbf{x}_t$ 的生成只取决于 $\mathbf{x}_{t-1}$ ，与更早的 $\mathbf{x}_0$ 无关。三个概率就此落定：

$$q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) = \frac{q(\mathbf{x}_{t-1} | \mathbf{x}_0) q(\mathbf{x}_t | \mathbf{x}_{t-1})}{q(\mathbf{x}_t | \mathbf{x}_0)}. \quad (4.7)$$

右边的三个条件概率分别是

- $q(\mathbf{x}_t | \mathbf{x}_{t-1})$ ：前向单步，见式 (3.2)，均值为 $\sqrt{\alpha_t} \mathbf{x}_{t-1}$ 、方差为 $\beta_t \mathbf{I}$ ；
- $q(\mathbf{x}_{t-1} | \mathbf{x}_0)$ ：把闭式 (3.8) 用在 $t-1$ 时刻，均值为 $\sqrt{\bar{\alpha}_{t-1}} \mathbf{x}_0$ 、方差为 $(1 - \bar{\alpha}_{t-1}) \mathbf{I}$ ；
- $q(\mathbf{x}_t | \mathbf{x}_0)$ ：把闭式 (3.8) 用在 t 时刻，均值为 $\sqrt{\bar{\alpha}_t} \mathbf{x}_0$ 、方差为 $(1 - \bar{\alpha}_t) \mathbf{I}$ 。

下面把式 (4.7) 的右边逐项算出来。分母 $q(\mathbf{x}_t | \mathbf{x}_0)$ 与 $\mathbf{x}_{t-1}$ 无关，先放到一边；分子是两个高斯密度之积，而每个高斯密度的指数都形如 $-(u - c)^2/(2\sigma^2)$ ，按 $u = \mathbf{x}_{t-1}$ 展开就是一个二次式。把两个指数展开、合并同类项，得到

$$q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) \propto \exp\left(-\frac{1}{2} a \mathbf{x}_{t-1}^2 + b \mathbf{x}_{t-1} + C\right), \quad (4.8)$$

其中

$$a = \frac{\alpha_t}{\beta_t} + \frac{1}{1 - \bar{\alpha}_{t-1}}, \quad (4.9)$$

$$b = \frac{\sqrt{\alpha_t}}{\beta_t} \mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}}{1 - \bar{\alpha}_{t-1}} \mathbf{x}_0, \quad (4.10)$$

常数 C 收集了所有不含 $\mathbf{x}_{t-1}$ 的项，配方时用不到它。 a 里的两项分别来自分子两个高斯的方差：方差越小，指数里的系数越大，乘积就越受它约束。

接下来对式 (4.8) 配方。回忆高斯密度 $\mathcal{N}(\mu, \sigma^2)$ 的指数展开后恰为 $-\frac{1}{2\sigma^2}u^2 + \frac{\mu}{\sigma^2}u + \text{常数}$ ，对照式 (4.8) 可以直接读出：方差是 $1/a$ ，均值是 b/a 。方差为

$$\tilde{\beta}_t = \frac{1}{a} = \frac{\beta_t(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}, \quad (4.11)$$

其中用了 $\alpha_t(1 - \bar{\alpha}_{t-1}) + \beta_t = 1 - \bar{\alpha}_t$ ：把 $\alpha_t = 1 - \beta_t$ 与 $\bar{\alpha}_t = \alpha_t \bar{\alpha}_{t-1}$ 代入左边展开即可验证。均值是把式 (4.10) 的 b 乘上方差 $\tilde{\beta}_t$ 、按分母通分整理得到：

$$\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t} \mathbf{x}_0 + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} \mathbf{x}_t. \quad (4.12)$$

把 $\tilde{\beta}_t$ 与 $\tilde{\boldsymbol{\mu}}_t$ 代回式 (4.8) 配方，指数重新聚成 $-\frac{(\mathbf{x}_{t-1} - \tilde{\boldsymbol{\mu}}_t)^2}{2\tilde{\beta}_t}$ 的形式，于是后验仍然是一个高斯：

$$q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}\left(\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t \mathbf{I}\right). \quad (4.13)$$

这个结果值得停下来看一眼：虽然 $q(\mathbf{x}_{t-1} | \mathbf{x}_t)$ 本身没有解析形式，但只要把起点 $\mathbf{x}_0$ 摆在桌面上，“往回走一步”的分布就完全确定——均值是 $\mathbf{x}_0$ 与 $\mathbf{x}_t$ 的加权组合，方差只由各步系数决定。然而采样时手里并没有 $\mathbf{x}_0$ ，它正是要生成的东西。于是还差最后一步改写。

4.3 改写均值

回看闭式 (3.8)： $\mathbf{x}_t$ 与 $\mathbf{x}_0$ 之间只差一个噪声项，所以可以反解出

$$\mathbf{x}_0 = \frac{1}{\sqrt{\bar{\alpha}_t}} (\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}). \quad (4.14)$$

这就是噪声的等效关系：“知道 $\mathbf{x}_t$ 里混的噪声”与“知道干净图片 $\mathbf{x}_0$ ”是一件事，二者通过这个式子一一对应。把它代入式 (4.12) 并整理，均值就不再依赖 $\mathbf{x}_0$ ，只依赖 $\mathbf{x}_t$ 和噪声：

$$\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \boldsymbol{\epsilon}) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon} \right). \quad (4.15)$$

采样时 $\boldsymbol{\epsilon}$ 是未知的，于是让网络直接预测它，记作 $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$ ，代回即得单步采样公式：

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}, \quad \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad (4.16)$$

其中 σ_t^2 取自式 (4.11) 的 $\tilde{\beta}_t$ (DDPM 实践中常直接取 $\sigma_t^2 = \beta_t$)。逐项读一遍：括号里是“把网络认为的噪声减掉，得到 $\mathbf{x}_0$ 的估计”； $1/\sqrt{\alpha_t}$ 把尺度回补到第 $t-1$ 步的量级； $\sigma_t \mathbf{z}$ 是反向过程自身保留的随机性，保证采样与训练时的前向过程匹配 ($t=1$ 时不再加这一项，直接输出 $\mathbf{x}_0$)。

网络在一步里干的事情可以画成图 7：上排是完整的采样链，从 $\mathbf{x}_T$ 一路去噪到 $\mathbf{x}_0$ ；虚线框里放大了其中一步——网络从 $\mathbf{x}_t$ 预测出噪声 $\boldsymbol{\epsilon}_\theta$ ，进而算出这一步高斯分布的均值 $\boldsymbol{\mu}_\theta$ ，再从 $\mathcal{N}(\boldsymbol{\mu}_\theta, \sigma_t^2 \mathbf{I})$ 中采出 $\mathbf{x}_{t-1}$ 。

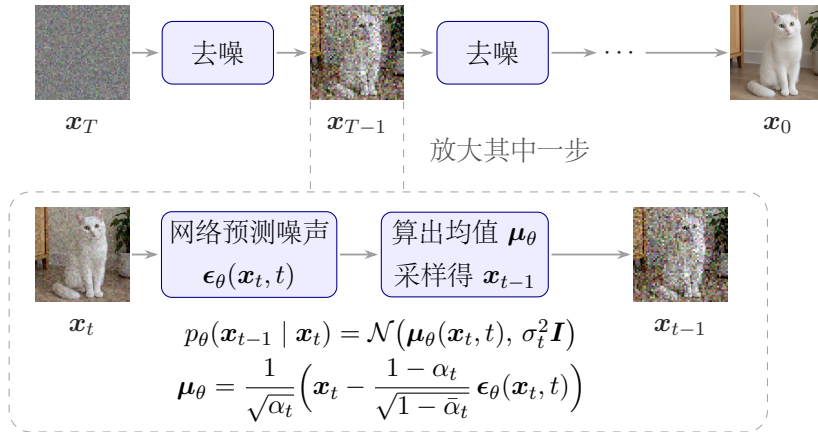


图 7: 反向计算框架

顺带回答一个常见疑问：为什么不直接让网络预测 $\mathbf{x}_0$ ？由式 (4.14)，两种做法只差一个确定的线性变换，数学上等价；但噪声的取值范围稳定、作为回归目标更好训练，所以实践中更常用预测噪声。

4.4 采样循环

把前面的结果合起来，采样就是一个很短的循环。此时手里已经齐了：网络给出单步反向分布 $p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t) = \mathcal{N}(\boldsymbol{\mu}_\theta, \sigma_t^2 \mathbf{I})$ ，均值按式 (4.15) 由噪声预测 $\boldsymbol{\epsilon}_\theta$ 算出。于是从纯噪声出发，反复“预测噪声、去掉一部分、往前走一步”即可：

1. 从标准高斯分布采出起点 $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ ，令 $t = T$ ；
2. 把当前的 $\mathbf{x}_t$ 和时间步 t 喂给网络，得到噪声预测 $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$ ；
3. 代入式 (4.16) 算出噪声更少一步的 $\mathbf{x}_{t-1}$ ，随后令 $t \leftarrow t - 1$ ；
4. 重复前两步；当 $t = 1$ 时不再加式 (4.16) 末尾的随机项，算出的就是最终结果 $\mathbf{x}_0$ 。

整个流程如图 8 所示。

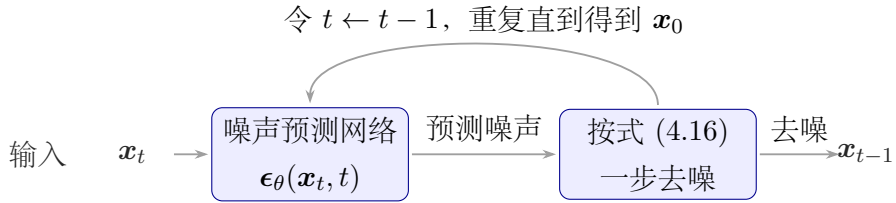


图 8: 采样循环

要注意，这个循环里既没有真实图片，也没有任何“对照答案”的步骤：每一步能用的信息只有当前的带噪样本 $\mathbf{x}_t$ 和时间步 t ，模型全靠训练时学到的那点经验去判断“这张图被加了多少噪声”。

最后把视角拉远，看整条生成链在概率上意味着什么（图 9）：上排是从 $\mathbf{x}_T$ 到 $\mathbf{x}_0$ 的完整采样链，每走一步都从单步反向分布里采一个样本；下排放大其中一步——网络去噪后给出这一步高斯分布的均值 $\boldsymbol{\mu}_\theta(\mathbf{x}_t, t)$ ，采出的 $\mathbf{x}_{t-1}$ 应当贴近这个均值。把链上每一步的概率连乘起来、对所有中间状态积分，就得到模型赋予一张完整图片的概率 $p_\theta(\mathbf{x}_0)$ ——它正是第 5 章训练目标想要最大化的对象。

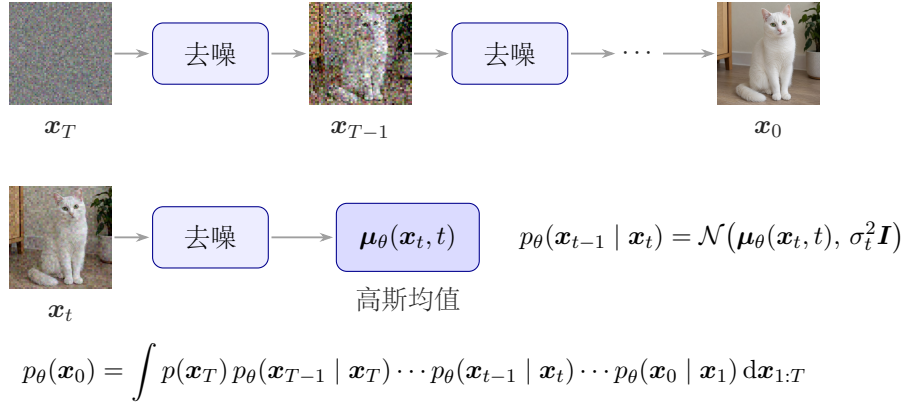


图 9: 生成全景

5 训练目标

训练时，先从数据集中取出真实样本 $\mathbf{x}_0$ ，随机采样时间步 t 和高斯噪声 ϵ ，再利用公式 (3.8) 构造带噪样本 $\mathbf{x}_t$ 。网络接收 $\mathbf{x}_t$ 和时间步 t ，输出噪声预测 $\epsilon_{\theta}(\mathbf{x}_t, t)$ 。

最常用的简化训练目标是噪声预测的均方误差：

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{\mathbf{x}_0, t, \epsilon} [\|\epsilon - \epsilon_{\theta}(\mathbf{x}_t, t)\|_2^2]. \quad (5.1)$$

5.1 损失来源

$\mathcal{L}_{\text{simple}}$ 并不是拍脑袋选的，它来自对似然下界（ELBO）的推导。我们真正想要的是让模型给出真实图片的概率尽量大，即最大化 $\log p_{\theta}(\mathbf{x}_0)$ ；但 $p_{\theta}(\mathbf{x}_0) = \int p_{\theta}(\mathbf{x}_{0:T}) d\mathbf{x}_{1:T}$ 这个积分算不动，于是转而最大化它的下界：

$$\log p_{\theta}(\mathbf{x}_0) \geq \mathbb{E}_{q(\mathbf{x}_{1:T} | \mathbf{x}_0)} \left[\log \frac{p_{\theta}(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T} | \mathbf{x}_0)} \right], \quad (5.2)$$

其中联合分布 $p_{\theta}(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^T p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t)$ 就是反向链搭出来的整个生成过程。从下界到三项分解只有两步代数：先把分子分母沿马尔可夫链逐项展开， $t=1$ 的两项合在一起，正是“由 $\mathbf{x}_1$ 还原 $\mathbf{x}_0$ ”的重建期望；再对 $t \geq 2$ 的每一项用贝叶斯公式 $q(\mathbf{x}_t | \mathbf{x}_{t-1}) = q(\mathbf{x}_t | \mathbf{x}_0) q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) / q(\mathbf{x}_{t-1} | \mathbf{x}_0)$ 改写，其中的 $q(\mathbf{x}_t | \mathbf{x}_0)$ 在相邻项之间首尾相消，剩下的项按“含 p_{θ} 的、

含先验 $p(\mathbf{x}_T)$ 的、其余”归成三组，下界就拆成三块：

$$\begin{aligned} \log p_\theta(\mathbf{x}_0) \geq & \underbrace{\mathbb{E}_q[\log p_\theta(\mathbf{x}_0 | \mathbf{x}_1)]}_{\text{重建项}} - \underbrace{D_{\text{KL}}(q(\mathbf{x}_T | \mathbf{x}_0) \parallel p(\mathbf{x}_T))}_{\text{先验匹配项}} \\ & - \sum_{t=2}^T \underbrace{\mathbb{E}_q[D_{\text{KL}}(q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t))]}_{\text{去噪匹配项}}. \end{aligned} \quad (5.3)$$

三项的含义：重建项要求 $\mathbf{x}_1$ 还能还原出 $\mathbf{x}_0$ ；先验匹配项要求加满噪声的 $\mathbf{x}_T$ 贴合标准高斯先验，这一项不含任何参数，是常数；真正训练网络的是去噪匹配项——每一步都在逼 $p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)$ 去贴合 4.2 节推出的后验。两个分布都是方差固定的高斯，KL 散度只取决于均值之差，而 p_θ 的均值按式 (4.15) 的形式由 ϵ_θ 给出、 q 的均值就是把 ϵ_θ 换成真值 ϵ ，于是每一项展开后恰是 $\|\epsilon - \epsilon_\theta\|_2^2$ （相差一些与网络无关的权重系数）。略去权重、把 t 换成随机采样，就得到 $\mathcal{L}_{\text{simple}}$ ——“预测噪声 + 均方误差”是 ELBO 逐项推导自然落下的结果，而非经验拼凑。

这个训练过程可以概括为：真实噪声是监督信号，带噪样本是输入，网络需要学会判断当前样本在当前噪声等级下被加入了什么噪声。训练完成后，模型便可以在没有真实样本的情况下，根据自己的噪声预测逐步采样。

- 输入：带噪样本 $\mathbf{x}_t$ 与时间步 t ；
- 监督信号：构造 $\mathbf{x}_t$ 时实际加入的噪声 ϵ ；
- 输出：模型预测的噪声 $\epsilon_\theta(\mathbf{x}_t, t)$ ；
- 损失：预测噪声与真实噪声之间的均方误差。

上述训练流程如图 10 所示。

6 网络结构

实际的噪声预测网络通常采用 U-Net 一类的结构，并通过时间嵌入把时间步 t 提供给网络。对于条件生成，还可以把类别、文字等条件编码后注入网络，结构示意图 11。

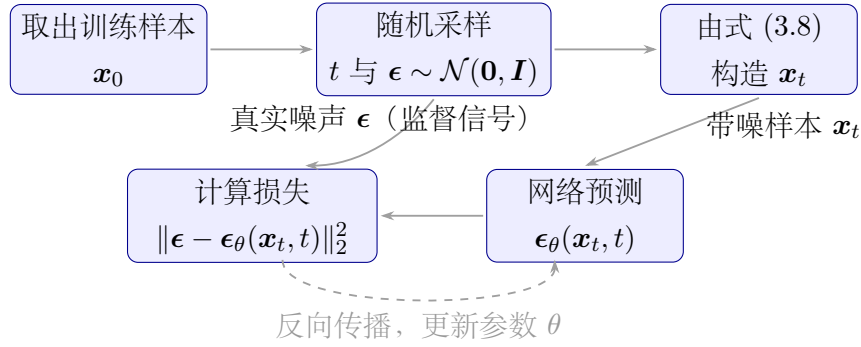


图 10: 训练流程

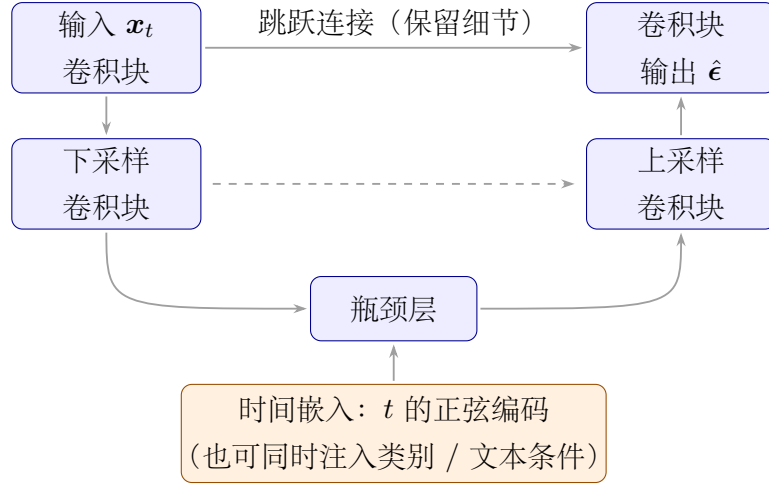


图 11: 网络结构

U-Net 的分工可以这样理解：下采样路径把图片逐步压缩，让网络在低分辨率上看到整张图的全局结构（噪声大概加在哪、整体布局是什么）；上采样路径再把分辨率逐步恢复回来；两路之间的跳跃连接把编码阶段保留的细节直接送进解码阶段，所以输出既能去噪、又不丢纹理。时间嵌入解决的是“同一个网络怎么区分不同噪声等级”的问题： t 先经过正弦编码变成一个向量，再注入每个残差块，相当于告诉网络“当前是加了 t 步噪声的版本，该减多少噪声”。做条件生成时，类别或文本编码也走这条注入通道，网络就能按条件往指定的方向去噪。